How APRS Chess sends a whole board in one packet

aprsd-chess-extension · position-code format v1 · generated from the code in `aprsd_chess_extension/codec.py`

1. The problem

An APRS message carries at most **67 characters** of text. aprsd cuts anything longer and never splits it. The text must be printable ASCII without the characters `|`, `~` or `{`, and aprsd replaces some words with ∗∗∗∗.

Way to describe a position	Size	Fits one message?
FEN, the standard text notation	56 to about 90 characters	Not reliably
The list of moves played	about 5 characters per half-move (a 40-move game is about 400)	No; it grows every move
Position code (this document)	**at most 46 characters**	**Always**

The server keeps each game's full move list. When a client needs the board, after a restart or when it reattaches with the game code, the server sends one *state line* that contains the position code:

```
3135 S N0AAA B 83BWVC1049RQWPACHG010024G0CSM7CY36D6MV02R5WWR
```

That line is 60 characters: the game code, `S` for "state", your opponent, your color, and the 45-character position code. The rest of this document decodes it bit by bit.

2. The bit layout

The position code is a string of bits, written 5 bits per character in Crockford base32. The fields come in this order:

Field	Bits	Meaning
version	2	format version, currently 1
result	2	0 ongoing, 1 White won, 2 Black won, 3 draw
draw offer	2	0 none, 1 offered by White, 2 offered by Black
occupancy	64	one bit per square, a1 first, then b1 … h1, a2 … h8; 1 = a piece is there
pieces	4 per piece	one 4-bit code for each occupied square, in the same square order
move number	12	the FEN full-move number
halfmove clock	8	moves since the last capture or pawn move (50- and 75-move rules)
last move	6 + 6	from-square and to-square, for highlighting (a1a1 = none)
padding	0 to 4	zero bits to finish the last character

Piece codes

Code	Piece	Code	Piece
0	♙ white pawn	6	♟ black pawn
1	♘ white knight	7	♞ black knight
2	♗ white bishop	8	♝ black bishop
3	♖ white rook	9	♜ black rook
4	♕ white queen	10	♛ black queen
5	♔ white king	11	♚ black king
12	rook that can still castle	13	pawn capturable en passant
14	black king, black to move	15	unused

> **Why there are three extra codes.** A FEN has fields for side to move, castling rights and en passant. Here they are folded into the pieces, so they cost no extra bits:
>
> - **Castling (12).** A rook that can still castle gets code 12 instead of 3 or 9. It can only be on a1, h1, a8 or h8, and the square gives its color. Code 12 on h1 means White can still castle king-side.
> - **En passant (13).** A pawn that can be captured en passant right now gets code 13. Only a pawn that just moved two squares can have it: on rank 4 it is White's, on rank 5 Black's.
> - **Side to move (14).** When it's Black's turn, the black king gets code 14 instead of 11. Otherwise White is to move.

Why it's always 46 characters or less

There are at most 32 pieces, so the code is at most 6 + 64 + 32×4 + 12 + 8 + 12 = **230 bits** = 46 characters. Fewer pieces make it shorter: with 30 pieces on the board, this example is 225 bits, or 45 characters.

The alphabet is `0123456789ABCDEFGHJKMNPQRSTVWXYZ`, which has no I, L, O or U. Every word aprsd masks contains one of those letters, so the filter can never alter a code. None of `| ~ {` is in the alphabet either. Decoding ignores case and also reads O as 0 and I or L as 1.

3. Worked example: decoding the state line

This code was captured from a real game played in testing, game 3135, after 20 half-moves. You (N0BBB) are Black, playing against N0AAA.

Step 1. Characters to bits

Each character stands for a number from 0 to 31, which is 5 bits. The first six:

Char	Value	Bits
8	8	01000
3	3	00011
B	11	01011
W	28	11100
V	27	11011
C	12	01100

All 45 characters give these 225 bits, colored by field:

```
010000001101011111001101101100000010000000100010011100010111111001011001010011001000110000
000000000100000000000000100010010000000000110011001101000011101100111100001100110011010011
0101001101100000000101100000101111001110011000
```

header occupancy pieces move number halfmove clock last move padding

Step 2. Header

`010000`: version `01` = 1, result `00` = 0 (game ongoing), draw offer `00` = 0 (none).

Step 3. Which squares have pieces

8	**1**	0	0	**1**	0	**1**	**1**	0
7	0	**1**	0	**1**	**1**	**1**	**1**	**1**
6	**1**	0	0	**1**	**1**	**1**	0	0
5	0	0	0	0	**1**	0	0	0
4	0	0	0	0	**1**	0	0	0
3	0	**1**	**1**	0	**1**	**1**	0	0
2	**1**	**1**	**1**	**1**	0	0	**1**	**1**
1	0	0	**1**	**1**	0	**1**	0	**1**
	a	**b**	**c**	**d**	**e**	**f**	**g**	**h**

The next 64 bits are one per square: a1 first, h8 last. Laid out as a board (rank 8 at the top), a 1 marks a square with a piece on it.

There are 30 ones, so 30 piece codes follow, one per blue square, in the same a1-to-h8 order.

Step 4. What each piece is

Each occupied square gets 4 bits. The first eight, which cover White's back rank and second rank:

Square	Bits	Code	Piece
c1	0101	5	♔ white king
d1	0011	3	♖ white rook
f1	0010	2	♗ white bishop
h1	0011	3	♖ white rook
a2	0000	0	♙ white pawn
b2	0000	0	♙ white pawn
c2	0000	0	♙ white pawn
d2	0100	4	♕ white queen

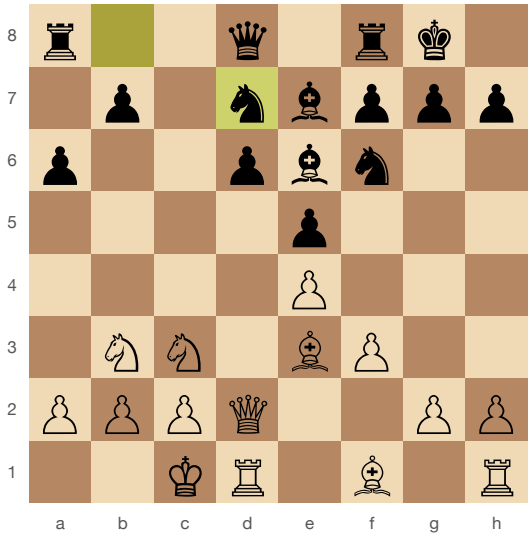
The other 22 are read the same way. White has castled queen-side and Black has castled king-side, so no rook carries code 12. No pawn can be captured en passant, so there's no code 13. The black king on g8 is a plain code 11, so **White is to move**.

Step 5. Move number, clock and last move

move number	`000000001011` = 11
halfmove clock	`00000101` = 5
last move	`111001` = b8, `110011` = d7: Black's knight from b8 to d7
padding	`000` (ignored)

The half-move number is 2 × (11 − 1) + 0 = **20**, plus 1 when Black is to move, which isn't the case here. That's how the client knows which move comes next, and how the server spots a stale or repeated move.

Result: the board



Putting it together gives this FEN:

```
r2q1rk1/1p1nbppp/p2pbn2/4p3/4P3/1NN1BP2/PPPQ2PP/2KR1B1R w - - 5 11
```

It matches the server's board exactly: the same pieces, White to move, no castling rights left, no en passant, halfmove clock 5, move 11. The last move, Nb8–d7, is highlighted.

The client draws this board and play goes on from here. The game's earlier moves aren't needed to keep playing; the server still has them for draw-by-repetition checks.

4. How long a sync takes over the air

This is the exchange measured when a freshly started client, with no saved data, synced game 3135. It is the same exchange the **Load game** button sends. It ran on a simulated 1200-baud channel: a direct path with no digipeaters or collisions, and 0.28 s of preamble per frame, the same as direwolf's `gen_packets`. Times are in seconds after the client sent its request:

Start	End	On air	From	Packet text	What it is
0.00	0.69	0.69	`N0BBB-6`	`chess 3135 sync{92`	client asks for the game
0.70	1.30	0.60	`N0SRV-12`	`ack92`	server acknowledges
1.30	2.30	1.00	`N0SRV-12`	`3135 S N0AAA B 83BWVC1049RQWPACHG010024G0CSM7CY36D6MV02R5WWR{7035`	state line
3.12	3.74	0.62	`N0BBB-6`	`ack7035`	client acknowledges

The board appears about 2.3 seconds after the request goes out. Most of that is the state line itself, about 1 second on air for 106 bytes. The client's final ack goes out afterwards; the board doesn't wait for it.

The file `docs/samples/sync-exchange-1200baud.mp3` has these four packets as real 1200-baud AFSK, with the timing above. direwolf's `atest` decodes all four from it.

> **On a real channel** the timing depends on the path:
>
> - Each digipeater hop sends the packet again. With the default WIDE1-1,WIDE2-1 path, expect each packet on air 2 or 3 times.
> - An APRS-IS round trip through igates adds about 1 to 3 seconds.
> - DigiPi's direwolf uses TXDELAY 300 ms and TXTAIL 100 ms, so each frame there takes about 0.1 s longer than in the table.
> - If a packet is lost, aprsd sends it again about 62 s after the first try and about 155 s after it, 3 tries in all. A lost state line therefore costs about a minute, not a few seconds.
>
> Even so, a sync is a single short exchange, while sending the move list would take several messages.

5. Where this lives in the code

`aprsd_chess_extension/codec.py`	`encode()` / `decode()`, the format described here
`aprsd_chess_extension/protocol.py`	the state line (`fmt_state`) and the client's parser (`parse_reply`)
`aprsd_chess_extension/server.py`	builds the state line from the server's move list
`tests/test_codec.py`	300 random games, every position round-tripped and checked to be ≤ 46 characters